

ניהול נתונים באינטרנט –

תרגיל בית 3

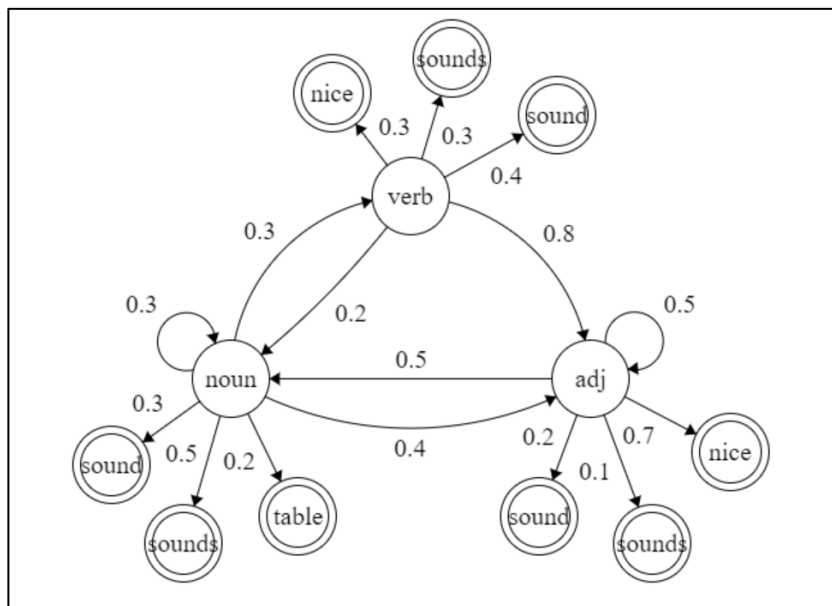
מגישי התרגיל:

- יעקב חזון – .
- עדי אלדר – .

תאריך הגשה: 11.06.2020.

שאלה 1:

נצייר תחילה את מכונת המצבים שלנו:



סעיף א':

כעת נחשב לפי אלגוריתם Viterbi את הנדרש עבור המשפט "sound sounds sound" (נסמן בהדגשה את המקסימום הנבחר בכל שלב):

State\prefix size	0	1	2
Noun	$0.4 * 0.3 = 0.12$	$0.5 * \max\{0.12 * 0.3, 0.08 * 0.2, \mathbf{0.08 * 0.5}\} = 0.02$	$0.3 * \max\{\mathbf{0.02 * 0.3}, 0.0108 * 0.2, 0.0064 * 0.5\} = 0.0018$
Verb	$0.2 * 0.4 = 0.08$	$0.3 * \max\{\mathbf{0.12 * 0.3}, 0.08 * 0, 0.08 * 0\} = 0.0108$	$0.4 * \max\{\mathbf{0.02 * 0.3}, 0.0108 * 0, 0.0064 * 0\} = \mathbf{0.0024}$
Adj	$0.4 * 0.2 = 0.08$	$0.1 * \max\{0.12 * 0.5, \mathbf{0.08 * 0.8}, 0.08 * 0.5\} = 0.0064$	$0.2 * \max\{0.02 * 0.4, \mathbf{0.0108 * 0.8}, 0.0064 * 0.5\} = 0.001728$

לאחר חישוב הטבלה, ניתן לראות כי הערך המקסימלי בשלב 2 הינו עבור המצב verb (0.0024). נחלץ את המסלול הסביר ביותר על ידי שחזור המסלול הנ"ל: $adj \rightarrow noun \rightarrow verb$ (כלומר, התיג הסביר ביותר למילה הראשונה הינו *adjective*, למילה השנייה *noun* ולשלישית *verb*).

סעיף ב':

כעת נחשב לפי אלגוריתם Viterbi את הנדרש עבור המשפט "sounds sound nice" (נסמן בהדגשה את המקסימום הנבחר בכל שלב):

State\prefix size	0	1	2
Noun	$0.4 * 0.5 = 0.2$	$0.3 * \max\{0.2 * 0.3, 0.06 * 0.2, 0.04 * 0.5\} = 0.018$	0
Verb	$0.2 * 0.3 = 0.06$	$0.4 * \max\{0.2 * 0.3, 0.06 * 0, 0.04 * 0\} = 0.024$	$0.3 * \max\{0.018 * 0.3, 0.024 * 0, 0.016 * 0\} = 0.00162$
Adj	$0.4 * 0.1 = 0.04$	$0.2 * \max\{0.2 * 0.4, 0.06 * 0.8, 0.04 * 0.5\} = 0.016$	$0.7 * \max\{0.018 * 0.4, 0.024 * 0.8, 0.016 * 0.5\} = 0.01344$

לאחר חישוב הטבלה, ניתן לראות כי הערך המקסימלי בשלב 2 הינו עבור המצב *adj* (0.01344). נחלץ את המסלול הסביר ביותר על ידי שחזור המסלול הנ"ל: *noun->verb->adj* (כלומר, התיוג הסביר ביותר למילה הראשונה הינו *noun*, למילה השנייה *verb* ולשלישית *adj*)

שאלה 2:

סעיף א' - הפיתרון מופיע בקובץ "question2a.py" המצורף. כאשר הפונקציה שמקבלת מערך של שמות קבצים ו-k הינה הפונקציה *fagin*. כפי שניתן לראות בקובץ, צירפנו מספר פונקציות עזר נוספות, וביניהן פונקציה שתעזור לנו לטעון את קבצי ה-csv עם הדירוגים ושמות הקבצים בתוכם. על מנת שנוכל להריץ בקלות את סעיף ב'. הפונקציה *fagin* אכן מחזירה את המבוקש בסעיף.

סעיף ב' - הפיתרון מופיע בקובץ "question2b.txt" המצורף. על מנת להריץ את הקובץ שבסעיף א' עם ערכי k שונים, יש לשנות ידנית את הפרמטר המועבר לפונקציה *fagin* בקוד.

שאלה 3:

$X_0X_1...X_l = \text{sentence}$

$N = \text{number of states}$

$\text{Temp} = \text{list}()$

For $s = 1,...,N$:

For $j = 1,...,k$:

$V_{0,s,j} = P_{\text{start}}(s) * P_{\text{out}}(s, X_0)$

$A_{0,s,j} = \text{null}$

For $i = 1,...,l$:

For $s1 = 1,...,N$

For $s2 = 1,...,N$:

For $j = 1,...,k$:

$\text{Temp.add}(P_{\text{out}}(s1, X_i) * V_{i-1,s2,j} * P_{\text{trans}}(s2, s1), (s2, j))$

$V_{i,s1} = [\text{max-top-k } \{\text{Temp.keys}()\}]$

$A_{i,s1} = [\text{Temp.value}() \text{ of corresponding coordinates}]$

$\text{Top-k} = \text{max-top-k}_i \{V_{i,s1}\}$

Fold back using $A_{j,s1}$ for each of the top k (as normal Viterbi)

הסבר: נבחין כי השינוי של האלגוריתם המתואר בפסאודו-קוד מעלה הינו בעצם שינוי פשוט של החזקת k הערכים המקסימליים לפי החישוב הרגיל מהעמודה הקודמת בכל תא בעמודה הנוכחית. נכונות הדבר נשענת על העובדה כי כל אחד מהמסלולים בעלי ההסתברות מקסימלית עבור תא מסוים (מסלול עד למילה מסוימת ותיג עבודה = תא) הינו המשך של אחד מההסתברויות המקסימליות בתאים בעמודה הקודמת. לכן, הוספנו מבנה של Temp ששומר את כל ההסתברויות עבור תא מסוים כאשר הן מחושבות על פי העמודה הקודמת, לאחר חישוב ההסתברויות אנו לוקחים את ה- k הסתברויות הגדולות ביותר (על פי אלגוריתם selection כלשהו נמצא את האיבר ה- n הגדול ביותר וניקח את כל ההסתברויות הגדולות ממנו) ואנחנו שומרים מצביעים לתאים המתאימים ב-A.

כך בעצם נחזיר את המסלולים עם ההסתברות הגבוהה ביותר בעמודה האחרונה על פי ההסתברויות וחזרה לאחור על ידי מבנה הנתונים של המצביעים.

הסיבוכיות זמן של האלגוריתם: $O(NK + N^2lK + NKNl) = O(N^2lK)$

הסבר סיבוכיות:

- NK - אתחול מבני הנתונים של ההסתברויות והמצביעים.
- N^2lK - מעבר על הערכים ושמירתם בתוך הלולאה על המילים והתיגים (Nl).
- $NKNl$ - לקיחת ה- top_k ההסתברויות ושמירתם בתוך הלולאה על המילים והתיגים (Nl).

שאלה 4:

סעיף א':

```
def numOfPossibleTrees(CFG G, sentence s=x1...xn):  
    for i=1,...,n:  
        for X non terminal in G:  
            
$$\pi(i, i, X) = \begin{cases} 1, & \text{if } X \rightarrow x_i \text{ in } G \\ 0, & \text{else} \end{cases}$$
  
        for l=1,...,n-1:  
            for i=1,...,n-l:  
                j = i + l  
                for all X non terminal in G:  
                    
$$\pi(i, j, X) = \sum_{\substack{X \rightarrow YZ \in R \\ s \in \{i, \dots, j-1\}}} \pi(i, s, Y) * \pi(s + 1, j, Z)$$
  
    Return  $\pi(1, n, \text{Root})$ 
```

סעיף ב':

נסמן את מספר הנון-טרמינלים של G ב-N ואת מספר המילים במשפט ב-n. לכן, סיבוכיות הזמן של האלגוריתם הינה: $O(n^3 N^3)$.

הסבר:

- אתחול הטבלה - $O(n * N)$.
- לולאה שעוברת על כל האורכי המשפט ונקודות התחלה שונות - $O(n^2)$, סוכמים על כל חוקי הגזירה והמילים בחלק מהמשפט שבו אנו עוסקים - $O(n * N^2)$ (שכן בשל שזוהי צורת חומסקי ניתן לחסום את מספר חוקי הגזירה ב- N^2).

סעיף ג':

def leftBranchCYK(PCFG G, sentence $s=x_1...x_n$):

For $i=1,...,n$:

For X non terminal in G:

$$\pi(i, i, X) = \begin{cases} q(X \rightarrow x_i) & \text{if } X \rightarrow x_i \text{ in } G \\ 0 & \text{else} \end{cases}$$

For $l=1,...,n-1$:

For X non terminal in G:

$$\pi(l, n-1, X) = \max_{X \rightarrow YZ \in G} (q(X \rightarrow YZ) * \pi(l, n-2, Y) * \pi(n-1, n-1, Z))$$

$$bp(l, n-1, X) = \arg \max_{X \rightarrow YZ \in G} (q(X \rightarrow YZ) * \pi(l, n-2, Y) * \pi(n-1, n-1, Z))$$

Return $\pi(1, n, \text{Root})$ and backpointers bp which allow recovery